
EEE5776. Applied Machine Learning



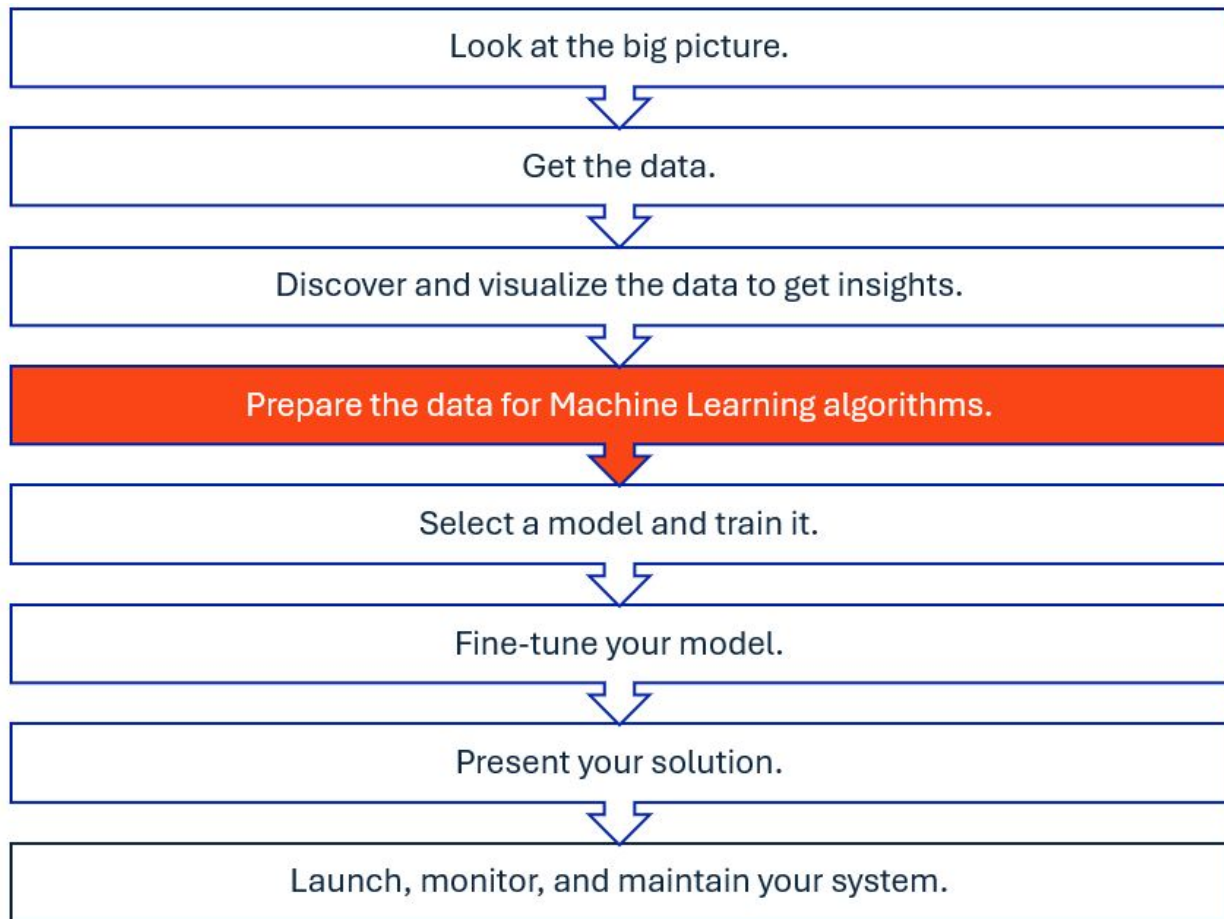
Module 2. Supervised Learning and Experimental Design

Feature Scaling

Transformation Pipelines

Linear Regression - Big Picture

End-to-End Machine Learning Process



A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps. The lines are of varying lengths and are arranged in a way that creates a sense of depth and movement.

Feature Scaling

Feature Scaling

One of the most important transformations you need to apply to your data is feature scaling. With few exceptions, Machine Learning algorithms don't perform well when the input numerical attributes have very different scales.

There are two common ways to get all attributes to have the same scale:

1. **Min-Max Scaling.** Values are shifted and rescaled so that they end up ranging from 0 to 1.
 - Highly affected by outliers.

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

X = original value

$X_{\min}$ = minimum value in the dataset (feature-wise)

$X_{\max}$ = maximum value in the dataset (feature-wise)

X_{scaled} = scaled value in the range [0,1] (or another specified range)

Feature Scaling

One of the most important transformations you need to apply to your data is feature scaling. With few exceptions, Machine Learning algorithms don't perform well when the input numerical attributes have very different scales.

There are two common ways to get all attributes to have the same scale:

1. **Standardization.** First it subtracts the mean value (so standardized values always have a zero mean), and then it divides by the standard deviation so that the resulting distribution has unit variance.
 - Less affected by outliers
 - Gaussianity assumptions

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

Feature Scaling - Important

- As with all the transformations, it is important to fit the scalers to the training data only, not to the full dataset (including the test set). Only then can you use them to transform the training set and the test set (and new data).
- Note that scaling the target values is generally not required.



Transformation Pipelines

Transformation Pipelines

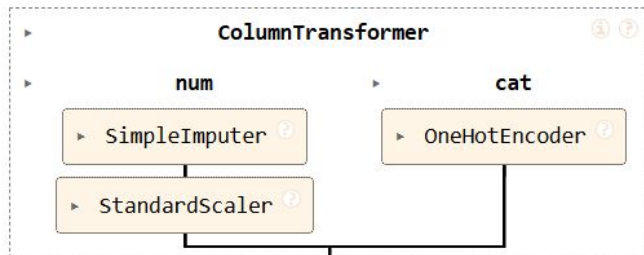
- Creating a `scikit-learn` transformation pipeline to scale and encode numerical and categorical features.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
```

```
# names of numerical attributes/features
num_attribs = np.setdiff1d(X_train.columns, ['ocean_proximity'])
# names of categorical attributes/features
cat_attribs = ['ocean_proximity']

# Pipeline for numerical attributes
num_pipeline = Pipeline([('imputer', SimpleImputer(strategy='median')),
                          ('std_scaler', StandardScaler())])
# Pipeline for categorical attributes
preprocessing = ColumnTransformer([('num', num_pipeline, num_attribs),
                                   ('cat', OneHotEncoder(), cat_attribs)])
```

preprocessing



Fit_transform and transform functions

```
X_train_prepared = preprocessing.fit_transform(X_train)
X_test_prepared = preprocessing.transform(X_test)
```

do NOT use fit_transform (or fit) using the TEST set.

```
X_train_prepared.shape, X_test_prepared.shape
```

```
((16512, 16), (4128, 16))
```

```
attribute_labels=np.hstack((num_attribs,
                             preprocessing.named_transformers_.cat.categories_[0]))
```

```
attribute_labels
```

```
array(['bedrooms_per_room', 'households', 'housing_median_age',
       'latitude', 'longitude', 'median_income', 'population',
       'population_per_household', 'rooms_per_household',
       'total_bedrooms', 'total_rooms', '<1H OCEAN', 'INLAND', 'ISLAND',
       'NEAR BAY', 'NEAR OCEAN'], dtype=object)
```

Fit_transform and transform functions

Method	Purpose	Learns Parameters?	Modifies Data?
fit()	Learns parameters (e.g., mean & std in scaling)	✓ Yes	✗ No
transform()	Applies the learned transformation	✗ No	✓ Yes
fit_transform()	Learns and applies the transformation in one step	✓ Yes	✓ Yes

The target

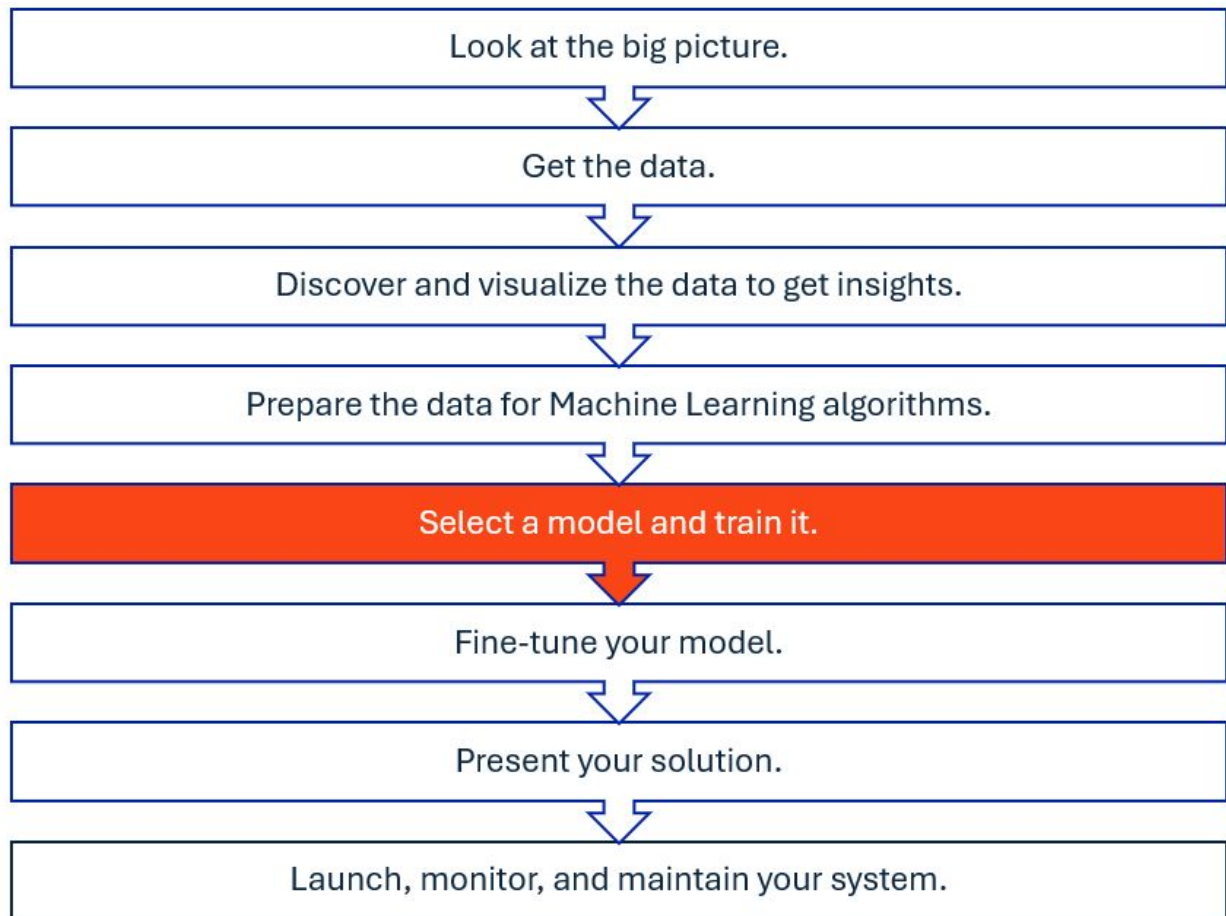
- Add something here

```
t_train
# generally do not scale the target values for regression tasks
# but we in some cases we want to scale/encode the target
# values for classification tasks
```

```
12655    72100.0
15502   279600.0
2908     82700.0
14053   112500.0
20496   238300.0
...
15174   268500.0
12661    90400.0
19263   140400.0
19140   258100.0
19773    62700.0
```

```
Name: median_house_value, Length: 16512, dtype: float64
```

End-to-End Machine Learning Process

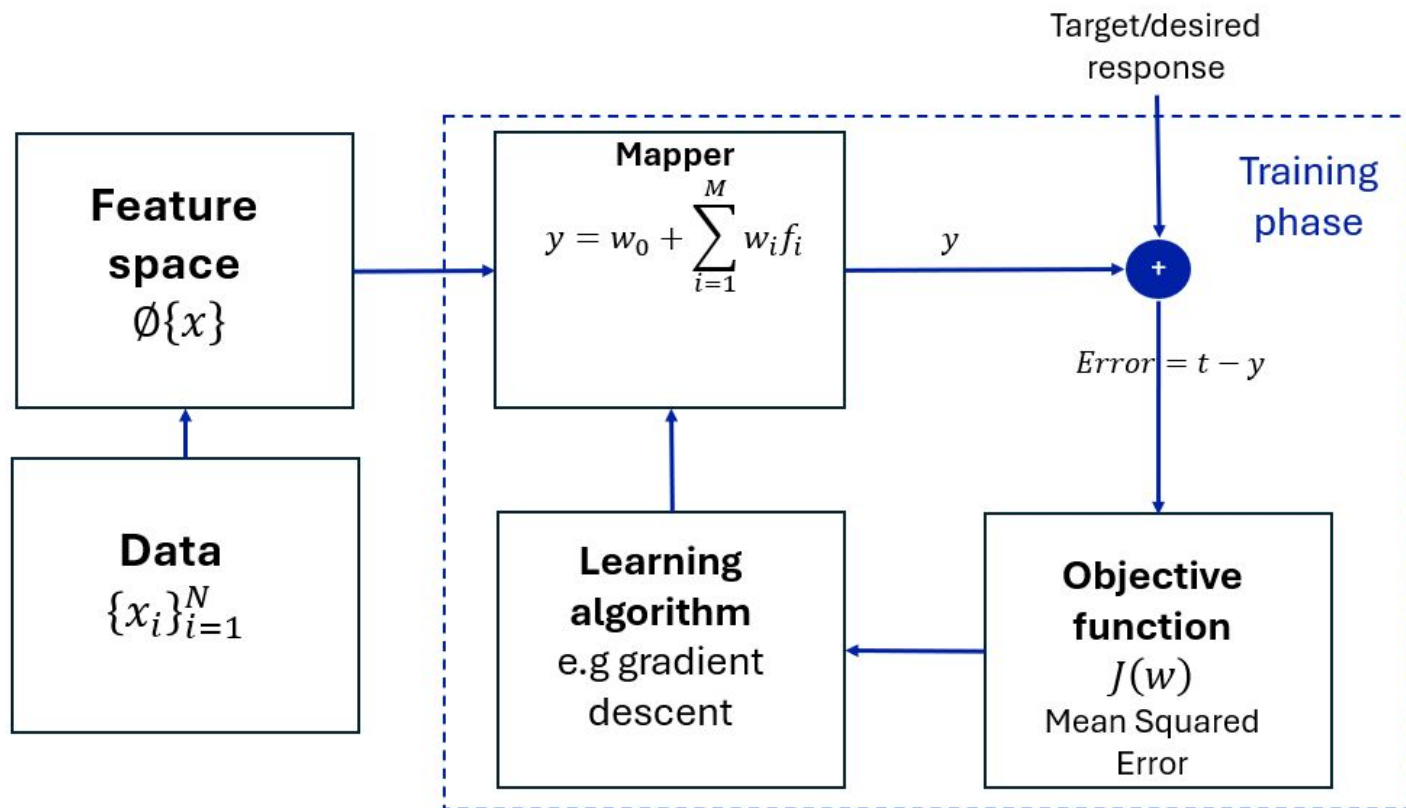


A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of parallel lines that curve and branch out.

Multiple Linear Regression

Multiple Linear Regression

The goal is to predict a single (continuous) outcome using several explanatory attributes/features.



Implementation - Fitting the model

```
from sklearn.linear_model import LinearRegression
# assumes the MSE objective function

# supervised learning
lin_reg = LinearRegression()

# fit function must have examples of input and target
lin_reg.fit(X_train_prepared, t_train)
```

Implementation - Discovering the model

```
# parameters w
```

```
lin_reg.coef_, lin_reg.coef_.shape
```

```
(array([ 16859.18615554,  42894.50706978,  13931.56938783, -56657.82876645,
        -56046.33896677,  78265.15994713, -46114.16368891,   913.38798131,
         7278.00685102,   322.80384089,   7996.0630126 , -17551.72508026,
        -51777.57825901, 105745.06664576, -22255.10163102, -14160.66167547]),
 (16,))
```

```
# intercept or bias term, w0
```

```
lin_reg.intercept_
```

```
np.float64(235539.02199526783)
```

Implementation - Analyzing the model

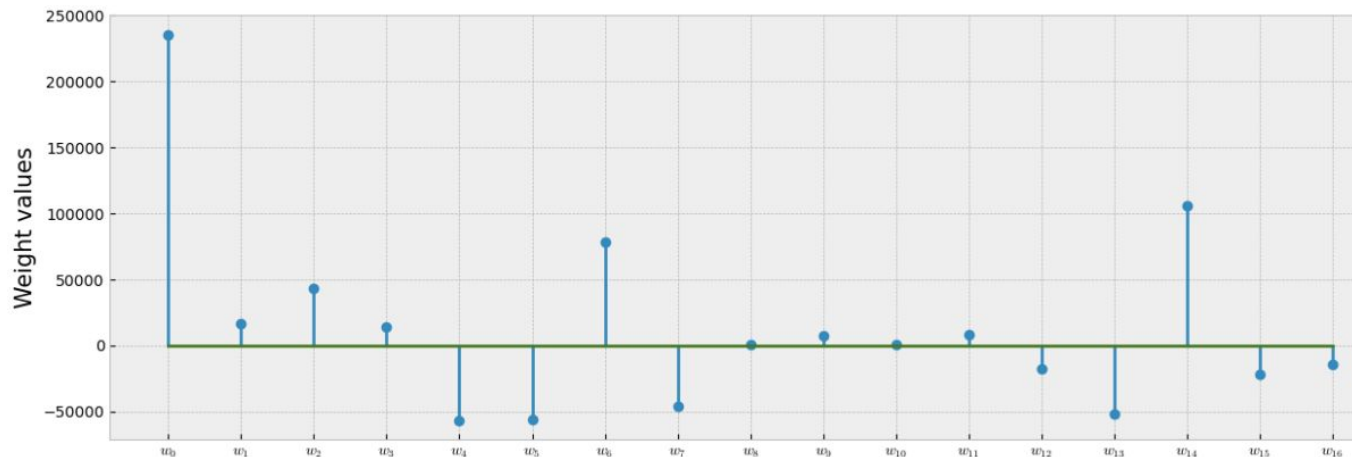
```
# concatenating all parameters into vector w
```

```
w = np.vstack((lin_reg.intercept_, lin_reg.coef_[ :, np.newaxis]))
w
```

```
array([[235539.02199527],
       [ 16859.18615554],
       [ 42894.50706978],
       [ 13931.56938783],
       [-56657.82876645],
       [-56046.33896677],
       [ 78265.15994713],
       [-46114.16368891],
```

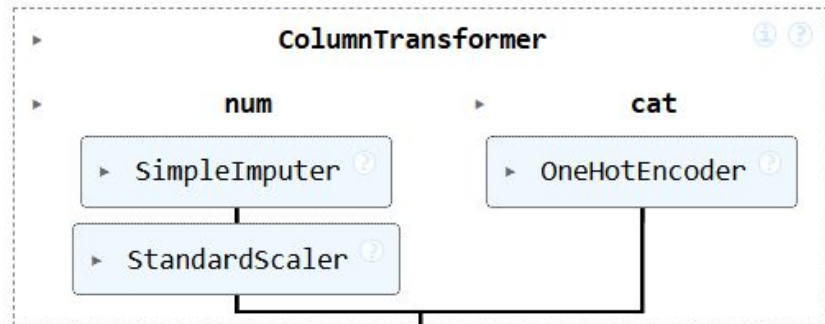


```
plt.figure(figsize=(15,5))
plt.stem(w)
plt.ylabel('Weight values', size=15)
plt.xticks(np.arange(len(w)), ['$w_{'+str(i)+'}$' for i in range(len(w))], rotation=0);
```

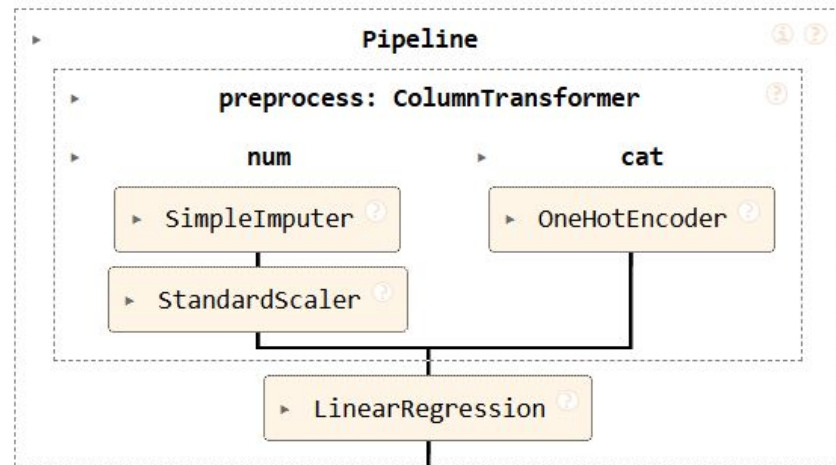


Implementation - Using Pipelines!

preprocessing

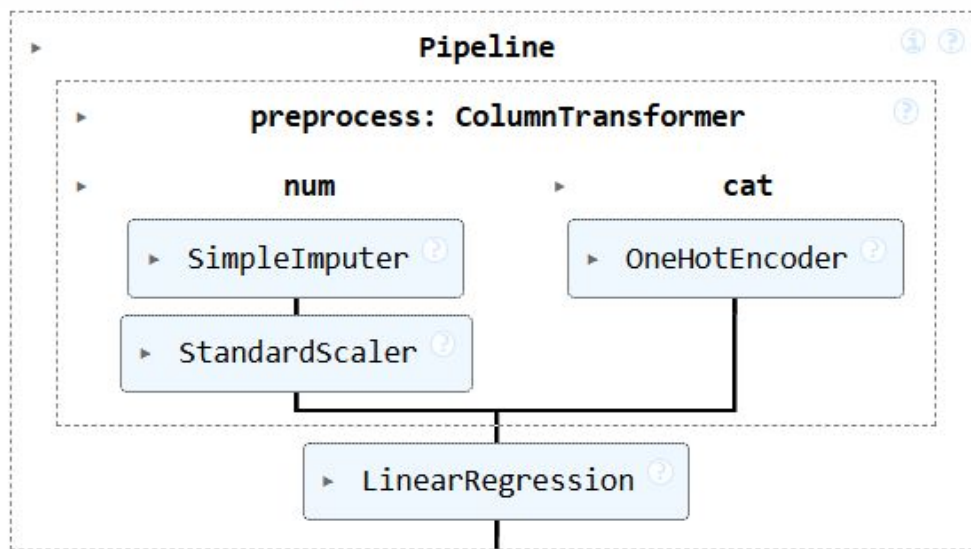


```
# Define pipeline
model = Pipeline([('preprocess', preprocessing),
                  ('lin_reg', LinearRegression())])
model
```



Implementation - Training

```
# Train pipeline
model.fit(X_train, t_train)
```



Implementation - Now Let's Predict!

```
# Making model predictions
y_train = model.predict(X_train)

y_train.shape

(16512,)

# Display predictions
plt.scatter(t_train, y_train)
plt.xlabel('True Target Values', size=15)
plt.ylabel('Predicted Target Values', size=15);
```



Implementation - Measures

We should summarize this information using quantitative performance measures. Examples include:

- Coefficient of determination (r^2) of the quantile-quantile (Q-Q) plot,
- Root Mean Squared Error (RMSE),
- Root Mean Absolute Error (RMAE),
- and others...

Implementation - Other Performance Measures - In-Practice

Performance of an algorithm can be determined using a variety of statistical goodness-of-fit measures.

- For regression tasks this includes error-based measurements, hypothesis tests, Q-Q plots.
- For classification tasks this includes error rate, accuracy, ROC curves, performance-recall curves.

But it can also be in terms of:

1. Risk,
2. Training time,
3. Training storage/memory,
4. Testing time,
5. Testing storage/memory,
6. Interpretability, namely, whether the method allows knowledge extraction which can be checked and validated by experts, and
7. computational complexity.

Implementation - Other Performance Measures - In-Practice

Performance of an algorithm can be determined using a variety of statistical goodness-of-fit measures.

- For regression tasks this includes error-based measurements, hypothesis tests, Q-Q plots.
- For classification tasks this includes error rate, accuracy, ROC curves, performance-recall curves.

But it can also be in terms of:

1. Risk,
2. Training time,
3. Training storage/memory,
4. Testing time,
5. Testing storage/memory,
6. Interpretability, namely, whether the method allows knowledge extraction which can be checked and validated by experts, and
7. computational complexity.

Using Scikit-Learn for metrics

```
from sklearn.metrics import mean_squared_error

mse_train = mean_squared_error(t_train, y_train)
rmse_train = np.sqrt(mse_train)

print('RMSE Training Set: ', rmse_train)
```

RMSE Training Set: 68173.59813802264



This is a very big value.

Let's try another model



```
from sklearn.tree import DecisionTreeRegressor

# Define pipeline
model2 = Pipeline([('preprocess', preprocessing),
                    ('dec_tree', DecisionTreeRegressor())])
```

```
# Train pipeline
model2.fit(X_train, t_train)
```

```
# Make model predictions
y_train = model2.predict(X_train)
y_test = model2.predict(X_test)
```

```
# Evaluate performance
mse_train = mean_squared_error(t_train, y_train)
rmse_train = np.sqrt(mse_train)

print('RMSE Training Set: ', rmse_train)
```

RMSE Training Set: 0.0

Suspiciously optimistically!

Next Steps

Underfitting vs. Overfitting

Since we should not touch the test data until we have finalized model selection, what should we do?

- We need to use part of the training set for training and part of it for model validation. This is known as **cross-validation**.